COMP2003 AI Use

Vibe Coding Evidence

By Daniel Dawdry

Dataset Cleaning

Prompt 1:

```
I am working on a university group project focused on AI in finance. I am responsible for the data preprocessing stage. Please generate a python function using pandas that loads a CSV file, removes rows with missing values, attempts basic numeric type conversion column by column, resets the index, and saves the cleaned data to a new CSV file. The code should be clearly commented.
```

What I've noticed is that this drops any row with a missing value which can remove lots of data. Another issue was it tries to convert every column which is slow. Other than these it seems to work. So only one more prompt is needed to sort this. Asking it to only drop rows where key columns are missing, and checking the column data type first and only changing it if it needs changing.

Prompt 2:

```
Improve this function by making the handling of missing data more efficient and using smarter column type conversion. The function should only try number conversion on appropriate columns and handle missing values in a controlled way while keeping the it simple and readable. The code should be commented and suitable as an improved version of the earlier code
```

Data Validation

Prompt 1:

```
Generate a simple and readable Python function that validates a pandas dataframe by checking if a list of required columns exists. The function should report any missing columns and be suitable for use as an early-stage data validation step in a data preprocessing pipeline.
```

Checks if needed columns exist before the pipeline runs. This will help prevent crashes later. The code it provided me does need a bit more commenting.

Prompt 2:

Can you generate comments onto this code in an easy to understand format to describe what's been done

Backend Helper

Prompt 1:

Generate a simple Python backend helper function that combines data_cleaning.py and dataset_validation.py. Meaning it loads a CSV file, validates required columns, applies the data cleaning function and saves the cleaned output. The function should be suitable as part of a basic data preprocessing pipeline.

This initial set of code again doesn't have comments. Asking to use detailed comments will give me a better understanding of what it's showing me and if it's suitable rather than blindly testing it.

Prompt 2:

Can you generate comments onto this code in an easy to understand format to describe what's been done

Basic Flask API endpoint

Prompt 1:

Create a simple Python Flask API called app.py for an AI finance prototype. The API should have one GET endpoint. The endpoint should accept a stock symbol as a query parameter. Use yfinance library to get historical data from between 2023 and 2024. The API should return the prediction as a JSON. This should contain a BUY, SELL, or HOLD signal.

3/3

Issues with output:

- Didn't handle missing symbols
- Didn't check if yfinance returned an empty field
- Didn't return proper HTTP status codes
- Didn't catch backend errors
- Too easy for API to crash

Prompt 2:

Improve the Flask API so i had basic error handling. If the user does not give a query parameter, return a JSON error message and HTTP status code 400. If yfinance returns an empt field return a JSON error message. Wrap data download in a try except block. If error happens return HTTP code 500. Keep the imports as they are.

2/2

Issues with output:

- Logic worked, but comments were unclear
- The file did not explain that it was a prototype
- The endpoint worked, but it was not obvious what each part was doing

Prompt 3:

add simple comments to flask API code so it is easier to understand. The comments should explain downloading stock data directly, adding features, generating prediction, catching errors

Backtesting Strategy

Prompt 1:

Create a function that simulates how much money a user would've made if they followed a basic trading rule over historical stock data. Function should start with 10000 in cash and zero shares. Make it loop through.

Issues with output:

- Loop started too early
- Didn’t prevent repeated buying while already holding shares
- Did not prevent selling when no position existed

Prompt 2:

Update the function so it starts looping from index 20, because the 20 day moving average needs enough data before it can be used. Use cash > 0 before buying, position > 0 before selling

Benchmark Comparison

Prompt 1:

Write a simple python function that acts as a benchmark strategy to compare against an AI trading model. The function should take a pandas field and should simulate investing a fixed starting amount of 10000 into a stock at the first closing price and holding until last closing price. Use pandas indexing. Calculate how many shares can be bought at the start and then calculate the final value at the end using last closing price

Issues with output:

- Calculated price difference but not number of shares
- Didn't simulate realistic investing behaviour
- The final value calculation was unclear

Prompt 2:

Update the function so it correctly simulates a real buy and hold strategy. To retrun the final value use: start_price = df.iloc[0]["Close"] and end_price = df.iloc[-1]["Close"]

Feature Engineering

Prompt 1:

Write a python function using the pandas library that takes a field and generates simple features that can be used for prediction. First calcualte daily returns as a percentage. Then calculate a short term moving average using a 5 day rolling window, and a longer term moving average using a 20 day rolling window. Function should return the updated field

Issues with output:

- Didn’t handle missing values created by rolling calculations
- Returned a field with NaN values which could break later
- Did not make sure data was clean before being passed to the model

Prompt 2:

> improve the add_features function so it removes rows with missing values after calculating features. Use dropna after creating returns. Make sure returns are using pct_change, ma_5 is using a 5 day rolling mean, ma_20 is using a 20 day rolling mean

Performance Evaluation

Prompt 1:

> Write a Python function that takes two inputs and calculates how well a strategy performed. It should return the total profit or loss and the percentage return. The output should be structured as a dictionary

Issues with output:

- Only calculated basic profit and percentage return
- Didn’t allow comparison of individual trades
- Didn’t include any measure of consistency like win rate

Prompt 2:

> Improve the calculate_performance function by adding an optional parameter. This should be a list of trade profits and losses. If it's given, calculate a win rate. The win rate should be the percentage of trades that resulted in profit. Use a simple apporach where trade is considered a win if its value is greater than zero

Issues with output:

- The structure of returned data wasn't clear
- It didn't include all results in a single output
- Some values were returned separately instead of grouped together

Prompt 3:

> Refine the calculate_perfoemance function so that all results are returned in a single dictionary. Make sure that win rate is only calculated if trade_results is given, otherwise return none

Signal Prediction Logic

Prompt 1:

> Write a simple python function that takes a pandas field that already contains calculated features such as moving averages. Use last row of field to generate a trading signal. If a short term moving average column is greater than a long term moving average column then return a BUY signal. If it's lower then return a SELL signal

Issues with output:

- Only returned the signal and no context
- Didn't include any confidence value
- Output format was too basic for API use

Prompt 2:

> Improve the predict_signal function so it returns both a signal and a confidence vlue. The output should be a dictionary with two keys: signal and confidence. Use BUY when ma_5 > ma_20, SELL when ma_5 < ma_20, HOLD when equal. Assign confidence values such as 0.75 for BUY, 0.70 for SELL, 0.50 for HOLD

API Response Formatting

Prompt 1:

Create a simple helper function that formats prediction results into a JSON response. The function should take two inputs, a trading signal and a confidence value. Use flasks jsonify function to return the result in JSON formt. Output should include: signal and confidence

2/2